
DARASA API: PRINCIPAL ENGINEERING BLUEPRINT (v2.0)

DOCUMENT STATUS: FINAL FOR ARCHITECTURAL REVIEW

SYSTEM DOMAIN: EDUCATIONAL ERP (KENYA CBC FRAMEWORK)

CORE ARCHITECTURE: EVENT-DRIVEN MODULAR MONOLITH ("THE CITADEL")

1. ARCHITECTURAL PRINCIPLES & PHILOSOPHY

Darasa API strictly adheres to the following software engineering principles to ensure maintainability, scalability, and legal compliance:

1. **Domain-Driven Design (DDD):** The codebase is partitioned by business capabilities (**finance**, **grading**, **academics**). Cross-domain database joins are strictly forbidden.
2. **Command Query Responsibility Segregation (CQRS):** Write operations (Teachers submitting grades) and Read operations (Principals viewing analytics) are separated at the database level to prevent locking under heavy load.
3. **Eventual Consistency (BASE over ACID across domains):** While localized transactions (e.g., tuition payment) use strict ACID database locks, cross-domain updates (e.g., updating a student's active status after payment) rely on Eventual Consistency via the Celery Event Bus.
4. **Privacy by Design:** Security and data protection are embedded into the architecture from inception, not bolted on as an afterthought, ensuring strict compliance with Kenyan data protection laws.

2. DISTRIBUTED SYSTEMS: OVERCOMING THE 8 FALLACIES

Shifting from a localized monolith to an asynchronous, event-driven system introduces network volatility. Darasa API mitigates these 8 fallacies using the following specific engineering solutions:

2.1 The Network is Reliable

- **The Reality:** Packets drop. External APIs (like Safaricom's Daraja) will timeout and resend webhooks.
- **The Darasa Solution:** Strict Idempotency. Every incoming request and internal Celery event is assigned a unique **Event_UUID**. The API checks a high-speed

Redis cache before processing. If the **Event_UUID** exists, the system returns a **200 OK** but drops the duplicate payload.

- Implementation: An **@idempotent(cache_ttl=86400)** Python decorator wrapping all critical write-endpoints.

2.2 Latency is Zero

- The Reality: Processing 50 CBC KNEC reports takes seconds. Mobile networks in rural Kenya have high latency.
- The Darasa Solution: Asynchronous Offloading & Optimistic UI. The Vue 3 client sends batch updates. The API immediately responds with **202 Accepted** and offloads the compilation to Celery worker queues. The Vue 3 client assumes success and updates the UI instantly, rolling back only if a WebSocket error is emitted.

2.3 Bandwidth is Infinite

- The Reality: Standard REST APIs suffer from over-fetching.
- The Darasa Solution: Backends for Frontends (BFF). We construct tailored endpoints (**/api/teacher-bff/fast-grid/**) that aggregate data from multiple domains on the server and transmit a mathematically compressed JSON payload to the client, minimizing payload weight.

2.4 The Network is Secure

- The Reality: Internal networks can be breached. Redis queues are vulnerable to packet sniffing.
- The Darasa Solution: Zero-Trust Event Bus. All JSON payloads pushed to Celery/Redis are cryptographically signed using a server-side HMAC-SHA256 key. Consumers verify the signature before execution.

2.5 Topology Doesn't Change

- The Reality: Docker containers die and scale dynamically.
- The Darasa Solution: Service Discovery. IP addresses are never hardcoded. Internal communication relies on Docker's native DNS resolution (e.g., **redis://redis-broker:6379/0**).

2.6 There is One Administrator

- The Reality: A school is a hierarchy of differing permissions.
- The Darasa Solution: Granular RBAC Middleware. Role-Based Access Control is enforced globally. A request is evaluated against a permission matrix mapped to the user's JWT before reaching the view layer.

2.7 Transport Cost is Zero

- The Reality: Passing large files (Lesson Plans, Portfolios) through the Django server consumes massive RAM and incurs high egress costs.
- The Darasa Solution: The Claim-Check Pattern. Clients request a presigned, temporary upload URL from Darasa. The Vue 3 client uploads the 10MB file directly to Cloudflare R2. Darasa only stores the resulting UUID string.

2.8 The Network is Homogeneous

- The Reality: Traffic originates from diverse, unpredictable devices.
 - The Darasa Solution: API Gateway Normalization. An NGINX reverse proxy terminates SSL, enforces strict rate-limiting, and normalizes headers before traffic hits the Django application servers.
-

3. RESILIENCE & CHAOS ENGINEERING PROTOCOLS

This section details how the system is engineered to handle inevitable hardware and software failures.

3.1 Data Armor & Protection

- End-to-End Data Protection: Data must be verifiable. When uploading to Cloudflare R2, the client generates a SHA-256 hash of the file. Cloudflare generates its own hash. If they diverge, the payload is rejected. Minor PII (e.g., medical records) is protected via Field-Level Encryption using `cryptography.fernet` at the Django ORM layer before writing to disk.
- Data Redundancy: PostgreSQL is configured with Write-Ahead Log (WAL) Streaming Replication. A secondary Read-Replica database guarantees a Recovery Point Objective (RPO) of < 1 second.
- Data Deduplication & Scrubbing: Background Celery cron jobs execute nightly to scrub orphaned data. Soft-deleted profiles are subjected to cryptographic shredding after 30 days to comply with "Right to be Forgotten" mandates.

3.2 Failure Modalities

- Crash-Only Software: Darasa's Celery workers are configured with `worker_max_tasks_per_child = 1000`. They intentionally terminate and reboot to prevent memory leaks, relying on Docker's orchestration to instantly spin up replacements.
- Design by Contract (DbC): Core calculation engines (e.g., CBC rubric compilers) enforce strict pre/post-conditions. Example: `assert 1 <= rubric_score <= 4`. If violated, the function triggers a Fail-Stop, halting the transaction to prevent database corruption.
- Fail-Fast vs. Failing Well:
 - *Fail-Fast*: If a request lacks a valid `Tenant-ID` header, the middleware drops it immediately with a `400 Bad Request`, expending zero database cycles.
 - *Failing Well (Graceful Degradation)*: If the external SMS gateway fails, the `disciplinary` module catches the timeout, trips a Circuit Breaker, and falls back to saving alerts strictly in the internal `Notifications` table.
- Fault Injection & Toxiproxy: During CI/CD, the `Toxiproxy` service intercepts the testing database connection and injects 1000ms latency and 5% packet loss. The automated test suite must prove the system can successfully process the Daraja API payment queue under these degraded conditions.

4. DOMAIN-DRIVEN DESIGN & SECURITY BLUEPRINT (APP-BY-APP)

4.1 **tenant/** (The Physical Boundary)

- Architecture: Schema-per-Tenant routing via **django-tenants**.
- Security Protocol: PostgreSQL Row-Level Security (RLS). A global connection variable is set per request. The database engine physically rejects queries trying to read schema data outside the bound tenant.
- TDD Blueprint: Schema Bleed Test. Instantiate **School_A** and **School_B**. Mock a concurrent high-load scenario using Python's **asyncio**. Assert that no HTTP request to **School_A** ever returns a database record belonging to **School_B**.

4.2 **core/** (Identity & Telemetry)

- Architecture: JWT-based stateless authentication with MFA SMS enforcement via Brevo/Africa's Talking.
- Security Protocol: Constant-Time String Comparison for all authentication checks to eliminate timing side-channel attacks.
- TDD Blueprint: Token Downgrade Test. Submit a JWT where the header specifies **alg="none"**. Assert the authentication engine rejects the token with a **401 Unauthorized**.

4.3 **finance/** (The Ledger)

- Architecture: Double-entry ledger interacting with external gateways (Daraja).
- Security Protocol: Pessimistic Locking. The database enforces **SELECT ... FOR UPDATE** on a student's ledger row when a payment webhook arrives, eliminating TOCTOU (Time of Check to Time of Use) race conditions.
- TDD Blueprint: Concurrency Bomb. Fire 10 simultaneous simulated Daraja M-Pesa webhooks for the exact same transaction ID. Assert the database locks successfully, processes 1, and rejects the remaining 9 without throwing a **500 Internal Server Error**.

4.4 **grading/ & academics/** (The Engine)

- Architecture: Decoupled structural data (Cohorts/Subjects) from volatile performance data (CBC Rubrics).
 - Security Protocol: Insecure Direct Object Reference (IDOR) Protection. All primary keys are UUIDv4. Middleware evaluates ownership: A teacher can only POST grades for a **Student_UUID** actively enrolled in their assigned **Cohort_UUID**.
 - TDD Blueprint: The Rubric Fuzz Test. Use the **Hypothesis** library to fire thousands of edge-case inputs (null characters, extremely large floats, SQL injection strings) at the Fast-Grid endpoint. Assert the API serializes and rejects all anomalies safely.
-

5. LEGAL & COMPLIANCE PLAN: KENYA DATA PROTECTION ACT (2019)

Darasa API processes the sensitive personal data of minors. Compliance is a hard architectural requirement, not a legal afterthought.

1. **Data Protection Impact Assessment (DPIA):** Prior to processing any high-risk data (e.g., disciplinary records, health conditions), a DPIA must be conducted to identify risks and implement safeguards.
2. **Verifiable Parental Consent:** The processing of children's personal data is prohibited without the explicit, verifiable consent of a parent or guardian. Darasa API utilizes the **core** module to require a digital consent signature upon a parent's first login.
3. **Age Verification:** The system must incorporate mechanisms for age verification, appropriate to the available technology and the risks involved. Darasa API addresses this by entirely removing direct access for minors; all platform access is restricted to adults (verified teachers and parents).
4. **Best Interests of the Child:** Article 53(2) of the Constitution and Section 8 of the Children Act mandate that the child's best interests are paramount in all processing activities. Data collection is strictly minimized, and profiling children for direct marketing purposes is physically blocked at the database level.
5. **Auditability (The Access Panopticon):** The system records every **GET** request directed at a student's profile into an append-only **AccessAuditLog**, fulfilling the DPA's transparency and accountability mandates.

Here is the consolidated, boardroom-ready master blueprint integrating the core Data Structures and Algorithms (DSA) curriculum directly into the distributed architecture, security protocols, and observability matrix of the Darasa API.

This document defines exactly how academic computer science translates into enterprise-grade resilience, database sharding, and legal compliance.

SECTION 8: APPLIED DATA STRUCTURES & DATABASE PHYSICS

We do not rely on Django's ORM to solve distributed systems problems. We implement raw mathematical structures to ensure predictable performance at a massive scale.

8.1 Hash Tables $\rightarrow$ Database Sharding & Tenant Routing

The Challenge: A single PostgreSQL instance cannot handle the I/O operations of 1,000 schools running end-of-term grade compilations simultaneously.

The Algorithmic Solution: Consistent Hashing Ring

- **Structure:** We abandon the naive modulo hash ($\text{tenant_id} \bmod N$) because adding a single database server forces a catastrophic re-indexing of all data. Instead, the hash space is mapped to a circular ring (e.g., 0 to $2^{32}-1$).
- **Execution:** Database nodes (Node A, Node B, Node C) are hashed onto the ring. When a request arrives for meruhigh.darasa.com, the API Gateway hashes the domain string, drops it onto the ring, and traverses clockwise to the nearest Node.
- **Security & Resilience:** If Node B fails, only the tenants cryptographically mapped to Node B are re-routed to Node C. 66% of the physical hardware experiences zero disruption. Cross-tenant routing requests must carry a signed JWT; if the signature fails, the hash lookup is aborted in $O(1)$ time, mitigating DDoS exhaustion attacks.

8.2 Graph Theory $\rightarrow$ CBC Curriculum & Prerequisite Mapping

The Challenge: The Kenyan CBC framework is a complex dependency matrix. Legacy systems treat it as a flat list, making it impossible to computationally track a student's foundational gaps.

The Algorithmic Solution: Directed Acyclic Graphs (DAG)

- **Structure:** The **curriculum** domain maps the entire KNEC syllabus as a DAG. Vertices represent Sub-strands (e.g., "Linear Equations"), and Edges represent prerequisites.
- **Execution (Topological Sort):** When the Darasa Compiler generates a term report, it runs a Depth-First Search (DFS) topological sort across the student's rubric scores.
- **Analytics Output:** The engine pinpoints the exact vertex where a student's understanding collapsed, generating NLP insights like: *"Failure to master Grade 7 Linear Equations (Vertex A) has cascaded into deficits in Grade 8 Quadratics (Vertex B)."*

To understand how this mathematical structure gives Principals and Teachers unprecedented insight into the syllabus, explore this visual representation of the CBC Curriculum DAG.

Show me the visualisation

8.3 Trees (Tries) $\rightarrow$ The RBAC Authorization Engine

The Challenge: Evaluating 500+ granular Role-Based Access Control (RBAC) rules against every incoming HTTP request using a flat array search ($O(n)$) introduces unacceptable latency.

The Algorithmic Solution: Prefix Trees (Tries)

- **Structure:** The NGINX/API Gateway holds a Trie of all secured endpoints in memory.
- **Execution:** A request for `/api/v1/finance/invoices/` traverses the Trie character by character or node by node (`api` $\rightarrow$ `v1` $\rightarrow$ `finance`).
- **Security Protocol:** Upon reaching the terminal node, the system executes a bitwise **AND** operation against the user's JWT Role Bitmask. The authorization check drops to $O(\log n)$ memory traversal, allowing the gateway to reject unauthorized requests before they ever wake up a Django thread.

8.4 Doubly Linked Lists $\rightarrow$ Temporal Client State (Undo/Redo)

The Challenge: Teachers rapidly inputting data into the Fast-Grid will make mistakes. Sending every **CTRL+Z** keystroke to the backend for state reversal floods the API and creates race conditions.

The Algorithmic Solution: Client-Side State Reversal

- **Structure:** The Vue 3 client maintains a localized Doubly Linked List mapping the last 15 JSON state changes in browser memory.
- **Execution:** Each node holds a snapshot pointer. Pressing **Undo** traverses the **prev** pointer. Pressing **Redo** traverses the **next** pointer.
- **Resilience:** Operations occur in strictly $O(1)$ time with zero network calls. The backend (**teacher-bff**) only receives the final, batched payload once the teacher commits the synchronized state.

8.5 Queues & Stacks $\rightarrow$ Eventual Consistency & Outbox Relays

The Challenge: Guaranteeing database writes and event emissions occur simultaneously without two-phase commit locking.

The Algorithmic Solution: The Transactional Outbox

- **Structure:** Database tables act as FIFO (First-In, First-Out) Queues.
- **Execution:** A synchronous database transaction commits the primary data and a localized **OutboxEvent** record simultaneously. A background Polling Relay operates as a stack-popper, fetching events in batches of 100 and dispatching them to the Redis message broker.
- **Security Protocol:** If Redis is unreachable, the event rests securely in the PostgreSQL queue. Poison-pill payloads that fail execution 3 times are routed to a strictly monitored Dead Letter Queue (DLQ).

SECTION 9: OBSERVABILITY & ALGORITHMIC CONTRACTS

Observability is the mathematical proof that our DSA implementations are functioning correctly in production. We enforce strict algorithmic contracts and telemetry pipelines.

9.1 Distributed Tracing & The Big O Enforcer

We implement OpenTelemetry to inject a cryptographic `Trace_ID` at the gateway, tracking execution cost across every micro-component.

- The $O(n^2)$ Regression Detector: The most lethal threat to Django is the $N+1$ query problem. If a developer accidentally loops over 50 students to fetch individual fee balances, execution time degrades to $O(n^2)$.
- The Observability Rule: The Datadog/Sentry APM actively monitors SQL query counts per HTTP cycle. If the engine detects >15 identical `SELECT` statements in a single trace, it triggers a catastrophic slack alert. The strict engineering fix mandates SQL `JOIN` operations (via `select_related`) to restore the query complexity to $O(1)$.

9.2 The Idempotency State Machine

Objective: Eliminate the "Network is Reliable" fallacy.

- Pre-Condition: Extract a deterministic unique identifier (e.g., Daraja `TransID`) or generate a SHA-256 hash of the JSON payload.
- Execution ($O(1)$ Lock): Perform a Redis `SETNX` (Set if Not Exists) operation with a 24-hour TTL.
- Post-Condition: If the lock is denied, the payload is a replay attack or network stutter. The system Fails-Fast, returning a safe `200 OK` to terminate the provider's retry loop, but physically drops the payload.

9.3 The Circuit Breaker Proxy

Objective: Prevent cascading system lockups during external dependency outages (e.g., Africa's Talking SMS API).

- State: CLOSED: Traffic flows freely. The proxy tracks failure rates. If consecutive timeouts > 5 , transition to `OPEN`.
- State: OPEN (Fail-Fast): Outbound connections are aborted instantly. The algorithm routes payloads to fallback data structures (e.g., `In_App_Notifications` table). A 60-second recovery timer initiates.
- State: HALF-OPEN: The timer expires. The proxy allows a single probe request. Success resets to `CLOSED`; failure snaps back to `OPEN`.

9.4 Asynchronous Queue Analytics

Celery queues are the blood pressure of the Darasa API.

- **The Telemetry Rule:** We continuously monitor the `length` of the Redis lists. If the `high_priority_compiler` depth exceeds 500 items, and the consumption rate (workers processing) drops below the publishing rate (teachers submitting), the Kubernetes autoscaler triggers.
- **Execution:** 3 additional stateless Celery pods are provisioned to absorb the spike, draining the queue back to equilibrium before spinning down.